TP N°1 : RÉGRESSION LINÉAIRE

L'objectif de ce TP est de :

- Définir le modèle de régression linéaire et expliquer son application
- Définir la fonction du coût et du gradient.
- Présentation des résultats sous forme graphiques et évaluation de modèle.
- Appliquer l'équation de régression à la prédiction.
- Présentation des résultats avec le module fitlm et comparaison avec la méthode normale.

Matériel :

- Matlab
- S'apporter votre PC.
- Liste de bases de données (ville.txt et salaires.txt)

1 Partie théorique

1.1 Descente du gradient

Dans cette section, vous allez ajuster les paramètres de régression linéaire θ à notre base de données en utilisant la descente du gradient

1.2 Mise à jour des paramètres du modèle

L'objectif de la régression linéaire est de minimiser la fonction de coût

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta \left(x^{(i)} \right) - y^{(i)} \right)^2 \tag{1}$$

où l'hypothèse $h_\theta(x)$ est donnée par le modèle linéaire

$$h_\theta(x) = \theta^T x = \theta_0 + \theta_1 x_1 \tag{2}$$

Rappelons que les paramètres de votre modèle sont les valeurs θ_j. Ce sont les valeurs que vous allez ajuster pour minimiser le coût $J(\theta)$. Une façon de le faire est d'utiliser l'algorithme itératif de descente du gradient. Dans cet algorithme, à chaque itération exécute une mise à jour.

$$\begin{aligned} \theta_j &:= \theta_j - \alpha \frac{\partial}{\partial \theta} J(\theta) \\ &:= \theta_j - \alpha \sum_{i=1}^{m} \left(h_\theta \left(x^{(i)} \right) - y^{(i)} \right) x^{(i)} \qquad \text{(M.à.J de } \theta_j \text{ pour toutes les } j\text{)} \end{aligned} \tag{3}$$

Une forme vectorielle de l'équation (3) est donnée par :

$$\theta_j^{new} := \theta_j^{old} - \frac{\alpha}{m} \times \left(X^{\mathrm{T}} * \left(X * \theta_j^{old} - y \right) \right) \tag{4}$$

À chaque étape de la descente du gradient, vos paramètres θ_j se rapprochent de plus en plus des valeurs optimales qui permettront d'obtenir le coût minimal $J(\theta)$.

Q.1 : À quoi sert la descente du gradient dans la régression linéaire ?

Q.2 : Démontrez l'équation (3), la dérivation de la fonction de coût de la régression linéaire.

2 Partie pratique

Dans ce TP, vous allez implémenter la régression linéaire à une variable pour prédire les bénéfices d'une chaîne de restaurants traditionnels. Supposons que vous êtes les gérants et que vous prévoyez d'ouvrir un nouveau point de vente dans différentes villes. La chaîne possède déjà de nombreux restaurants dans différentes villes et vous disposez de données sur les bénéfices et la population de ces villes. Le fichier ville.txt contient la base de données de notre problème de régression linéaire. La première colonne est la population d'une ville et la seconde est le bénéfice d'un restaurant dans cette ville.

2.1 Affichage des données

Avant de commencer une tâche, il est souvent utile de comprendre les données en les visualisant. Pour la base de données ville.txt, vous pouvez utiliser un nuage de points afin de visualiser les données, puisqu'il n'y a que deux propriétés à représenter (le bénéfice et la population). (De nombreux problèmes que vous rencontrerez dans la vie réelle sont multidimensionnels et ne peuvent pas être représentés sur un graphique à deux dimensions). Dans TP1.m, la base de données est importée à partir du fichier de données sous forme de variables X et y. :

```
%% Initialisation
clear ; close all ; clc

data = load('ville.txt'); % lire des donnees separees par des
   virgules
X = data(:, 1); y = data(:, 2);
m = length(y); % nombre d'exemples d'apprentissage
plot(X, y, 'bo', 'MarkerSize', 11); % Affichez les donnees
ylabel('Benefice en 10000$'); % Definir le label de l'axe des y
xlabel('Population de la ville en 10 000 habitants'); % Definir le
   label de l'axe des x
```

2.2 Implémentation

Dans TP1.m, nous avons déjà configuré les données pour la régression linéaire. Dans les lignes ci-dessous, nous ajoutons une autre dimension à nos données pour tenir compte du terme d'interception θ_0. Nous initialisons également les paramètres initiaux à 0 et le taux d'apprentissage alpha à 0,01.

```
X = [ones(m, 1), data(:,1)]; % Ajouter une colonne de "uns" a X
theta = zeros(2, 1); % Initialiser les parametres d'ajustement
iterations = 1500;
alpha = 0.01;
```

R [Note d'implémentation :] Nous stockons chaque exemple comme une ligne dans la matrice X. Pour prendre en compte le terme d'interception (θ_0), nous ajoutons une première colonne supplémentaire à X et lui attribuons tous des uns. Cela nous permet de traiter θ_0 comme une autre "caractéristique-feature".

2.2.1 Calculer le coût $J(\theta)$

En effectuant la descente du gradient pour apprendre à minimiser la fonction de coût $J(\theta)$, il est intéressant de surveiller la convergence en calculant le coût. Dans cette section, vous allez implémenter une fonction permettant de calculer $J(\theta)$ de façon à pouvoir vérifier la convergence de votre algorithme.

Votre prochaine tâche consiste à compléter le code du fichier CalulCost.m, qui est une fonction permettant de calculer $J(\theta)$. Rappelez-vous que les variables X et y ne sont pas des valeurs scalaires, mais des matrices dont les lignes représentent les exemples de l'ensemble d'apprentissage.

```
function J = CalculCost(X, y, theta)
%CalculCost Calcule le cout de la regression lineaire
% J = CalulCost(X, y, theta) determine le cout de l'utilisation de
   theta comme parametre de regression lineaire
% afin d'ajuster les points de donnees en X et y
% Initialiser quelques valeurs utiles
m = length(y); % nombre d'exemples d'apprentissage
% Vous devez retourner correctement les variables suivantes
J = 0;
% ====================== VOTRE CODE ICI ======================
% Instructions : Calculer le cout d'un choix particulier de theta
% Vous devez fixer J au cout.
h_theta =... ; %Completer la ligne en vous servant de l'eq(2).
sum_sq = sum(....); %Completer la ligne en vous servant de l'eq(1).
J = sum_sq*1/(2*m);
% =============================================================
end
```

Q.1 : Après avoir terminé la fonction, appelez cette fonction sur le programme initial TP1.m, et affichez la valeur de coût J.

Q.2 : Répétez le même calcul pour $\theta = [\theta_0; \theta_1] = [-1; 2]$, Que remarquez-vous ?

2.2.2 Descente du gradient

Maintenant, vous allez implémenter la descente du gradient dans le fichier DesctGrad.m. La structure de la boucle a été écrite pour vous, et vous n'avez qu'à fournir les mises à jour de θ à chaque itération. Lorsque vous programmez, assurez-vous de comprendre ce que vous essayez d'optimiser et ce que vous mettez à jour. N'oubliez pas que le coût $J(\theta)$ est paramétré par le vecteur θ, et non par X et/ou y. Autrement dit, nous minimisons la valeur de $J(\theta)$ en changeant les valeurs du vecteur θ, et non en changeant X et/ou y (Se référer à l'équation (3)) .

```
function [theta, J_hist] = DesctGrad(X, y, theta, alpha, iterations)
% DesctGrad Effectue une descente du gradient pour apprendre le theta
% Initialiser quelques valeurs utiles
m = length(y); % nombre d'exemples d'apprentissage
J_hist = zeros(iterations, 1);
```

```
6
7  for iter = 1:iterations
8  % ====================== VOTRE CODE ICI ======================
9  %   Instructions : Effectuer une seule tape de gradient sur le
       vecteur parametre theta.
10 %%L'impl mentation vectorielle de la descente du gradient selon l'Eq
       (4) est :
11 theta = ... - ..........;
12 % ============================================================
13 % Sauvegarder le cout J a chaque it ration
14 J_hist(iter) = CalculCost(X, y, theta);
15 end
16 end
```

Q.1 : Une fois que vous avez terminé, donnez les valeurs d'ajustement θ_0 et θ_1 ainsi que el coût $J(\theta)$ aprés la convergence.

Q.2 : Vous pouvez utiliser vos paramètres finaux dans TP1.m pour tracer la droite de la régression linéaire.

Q.3 : Vos valeurs finales pour θ seront également utilisées pour effectuer des prédictions sur les bénéfices dans des zones de 40000 et 80000 habitants. Donnez ces prédictions, en servant l'exemple :

```
predict1 = [1, 4] * theta*10000;
predict2 = [1, 8] * theta*10000;
```

Q.4 : Tracez la courbe $J(\theta) = f(it)$, le coût en fonction le nombre d'itération et déduisez le nombre d'itération nécessaire afin d'assurer la convergence (plot(1 :numel(J_his), J_his, '-b', 'LineWidth', 2);)

Q.5 : Quel est l'intéret de fixer un nombre des itérations ?

Q.6 : Appliquer la décente du gradient sur les 3 initialisations des paramètres (thetas) :

(a) Initialisation aléatoire
(b) Initialisation à zero
(c) Initialisation à un

Q.7 : Est-ce que l'initialisation des paramètres affecte la convergence ?

2.3 Régression linéaire par la fonction fitlm

Matlab propose une méthode plus simple d'ajustement des modèles linéaires : la fonction fitlm. Pour utiliser fitlm, nous commençons par placer nos données dans un modèle :

```
mdl = fitlm(X,y)  %Ajustez un modele de regression lineaire en
    utilisant fitlm.
```

Q.1 : Tracez la droite de la régression linéaire (plot(mdl)).

	Estimé (Estimate)
Interception (θ_0)	—
x_1	—
x_2 (θ_1)	—

Le modèle affiche également les informations sur les coefficients estimés, qui sont stockées dans la propriété Coefficients (mdl.Coefficients). Les resutats sont donnés par :

Q.2 : Donnez les valeurs d'ajustement θ_0 et θ_1, et comparez les avec ceux trouver dans la section 2.2.2. Que remarquez-vous ?

3 HomeWork

En tant qu'employé, vous devez connaître plusieurs facteurs qui peuvent affecter votre salaire. L'un de ces facteurs est le nombre d'années d'expérience. En effet, si vous avez plus d'expérience dans un domaine, vous pouvez obtenir un meilleur salaire. Dans ce cas, vous réaliserez quelques prédictions en utilisant la base de données salaires.txt qui couvrait le lien entre l'année d'expérience et le salaire.

Q.1 : Reprenez la partie 2.2.2. Ensuite, donnez la prédiction du salaire d'un employé dans 4 ans, et dans 8 ans.

Q.2 : Reprenez la partie 2.3.

4 Tâches optionnelles

Dans cette section, nous avons inclus quelques codes Python™ (facultatifs). Ils vous aideront à approfondir votre compréhension du TP, et si vous êtes en mesure de le faire, nous vous encourageons à les compléter également.

4.1 Affichage des données

[1]:
```
# chargement de bibliotheques
# Calcul scientifique et vectoriel en python
import numpy as np
# utilisé pour manipuler les chemins de répertoire
import os
# bibliothèque permet de tracer et de visualiser des données sous forme de graphiques.
import matplotlib.pyplot as plt
%matplotlib inline
datafile = os.path.join('ville.txt')
data = np.loadtxt(datafile, delimiter=',')
# Afficher les donnees
X, y = data[:, 0], data[:, 1]
# nombre d'exemples d'apprentissage
m = y.size
plt.scatter(X, y)
plt.xlabel('Population de la ville en 10000 habitants')
plt.ylabel('Benefice en 10000$')
```

4.2 Implémentation

[2]:
```
# Ajouter une colonne de uns à X
X = np.stack([np.ones(m), X], axis=1)
# Initialiser les parametres d'ajustuments
theta = np.array([0, 0])
```

4.2.1 Calculer le coût $J(\theta)$

[3]:
```
# =VOTRE CODE ICI ==
def CalculCost(x, y, theta):
m = len(y)
cout = np.sum(....)
return (1 / (2*m)) * cout
```

[4]:
```
# Test de la fonction cout
theta = np.array([0, 0])
CalculCost(X, y, theta)
```

4.2.2 Descente du gradient

[5]:
```
# Descente du gradien
def DesctGrad(x, y, theta, alpha, iters):
# Initialiser quelques valeurs utiles
m = len(y)
cout = np.zeros(iters)
theta = theta.copy()
# =VOTRE CODE ICI ==
for i in range(iters):
error = ....dot(...) - y
theta = .... - (alpha/m) * error.dot(x)
cout[i] = CalculCost(x, y, theta)
return theta, cout
```

[6]:
```
# Test de Descente du gradien
alpha, iters = 0.01, 1500
theta, cout = DesctGrad(X, y, theta, alpha, iters)
theta
```

[7]:
```
# La prédiction
prdct1 = np.array([1,4]).dot(theta)
prdct2 = np.array([1,8]).dot(theta)
prdct1,prdct2
```

[8]:
```
theta, cout = DesctGrad(X, y, theta, alpha, iters)
plt.scatter(X[:,1], y, label='Donnees d apprentissage')
plt.plot(X[:,1], X.dot(theta), 'r-', label='Regression Lineare')
plt.legend()
plt.xlabel('Population de la ville en 10000 habitants')
plt.ylabel('Benefice en 10000$')
```

[9]:
```
# Affichage de coût
plt.plot(np.arange(iters), cout)
plt.xlabel('Iterations')
plt.ylabel('Cout')
```

4.3 sklearn.linear_model

[10]:
```
from sklearn.linear_model import␣
 ↪SGDRegressor
sgd_reg =␣
 ↪SGDRegressor(max_iter=15000,␣
 ↪tol=1e-3, penalty=None, eta0=0.
 ↪01)
sgd_reg.fit(X, y)
```

[11]:
```
y_prdct = sgd_reg.predict(X)
plt.plot(X[:, 1], y_prdct, "r-")
plt.plot(X[:, 1], y, "b.")
plt.xlabel('Population de la ville␣
 ↪en 10000 habitants')
plt.ylabel('Benefice en 10000$')
plt.show()
```